FEAT-ZD-48600 · DE-01 Checking the coding prerequisites ■ · Repository (and Databricks) availability and write access

Data Engineering agent · item task.repo_access · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.repo_access.md · approved by Data engineer sim.data_engineer

task.repo_access: FEAT-ZD-48600

DE-01 Checking the coding prerequisites for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This step checks that the repository is available, that a working branch exists and that the agent identity can write to it. It also checks whether Databricks is needed. No code was changed in this step.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-01 Checking the coding prerequisites	record:brief
Owner / gate	Data engineer / automated checks, independent judge, sampled human evaluation	record:brief
Inputs	feat.tasks (b0fd4d2), org.engineering_stack (76d03ce)	record:brief
Lane tasks	TSK-01 (STORY-01) and TSK-02 (STORY-02), both in repository ap_pipeline	feat.tasks (section 1)

1. Outcome

The prerequisites are met. The ap_pipeline repository is available locally, and branch `feature/story` exists. The agent identity has write access. Databricks is not needed: no repository uses Databricks or a database, so this lane needs no Databricks workspace.

Check	Result	Evidence	Source
Databricks access	Not applicable: no repository uses Databricks or a database	The engineering stack and the tool view both say no Databricks workspace is used	org.engineering_stack; tool:repo-access
Repository available	Pass: ap_pipeline is available locally	The tool view reports the repository available locally. The checkout's repository memory, CI script and README were read	tool:repo-access; file:ap_pipeline/CLAUDE.md; file:ap_pipeline/ci.py; file:ap_pipeline/README.md
Repository creation needed	No: the repository already exists. Remote `origin` is set	`.git/config` remote `origin` → `https://github.com/arghya05/ap-pipeline-sample.git`	file:ap_pipeline/.git/config
Working branch	Pass: `feature/story` was created from `dev` at `3fcbe1b` and is checked out	`.git/HEAD` → `refs/heads/feature/story`; `feature/story`, `dev` and `main` all point to `3fcbe1bad11e1fb6b44523c7ca795d0c1ca7b1d8`	tool:repo-access; file:ap_pipeline/.git/HEAD; file:ap_pipeline/.git/refs/heads/feature/story; file:ap_pipeline/.git/refs/heads/dev; file:ap_pipeline/.git/refs/heads/main
Write access	Pass: the agent identity has write access, and the working tree is clean	Tool view	tool:repo-access

Check	Result	Evidence	Source
Branch model	Matches the convention: `main` (released), `dev` (integration), `feature/<story>`	Local branches `main`, `dev` and `feature/story` exist	org.engineering_stack; file:ap_pipeline/CLAUDE.md; feat.tasks (section 2); file:ap_pipeline/.git/refs/heads/main; file:ap_pipeline/.git/refs/heads/dev; file:ap_pipeline/.git/refs/heads/feature/story
Language and CI	Python 3.9; CI command is `/usr/bin/python3 ci.py`; it requires line coverage >= 90% and runs static checks	`ci.py` sets `COVERAGE_MIN = 90.0`	org.engineering_stack; file:ap_pipeline/ci.py
Current version	`0.4.0` (TSK-01 owns the version step)	`src/ap_pipeline/__init__.py`	file:ap_pipeline/src/ap_pipeline/__init__.py; feat.tasks (section 2)

2. Repository facts for the coding steps

Item	Value	Source
Ingestion module	`src/ap_pipeline/ingest.py`: `parse_feed(csv_text)` reads the CSV feed with a header	file:ap_pipeline/src/ap_pipeline/ingest.py
Transformation module	`src/ap_pipeline/transform.py`: `to_model(rows)` returns `(records, rejected)`; a rejected row's reason is the exception class name	file:ap_pipeline/src/ap_pipeline/transform.py
Data-quality module	`src/ap_pipeline/dq.py`: `quality_report(records, rejected)`	file:ap_pipeline/src/ap_pipeline/dq.py
Pipeline run	`src/ap_pipeline/pipeline.py`: `run(csv_text)` runs ingest, then transform, then the data-quality report	file:ap_pipeline/src/ap_pipeline/pipeline.py
Version file	`src/ap_pipeline/__init__.py` (`__version__ = "0.4.0"`)	file:ap_pipeline/src/ap_pipeline/__init__.py
Schedule (no change in this feature)	`src/ap_pipeline/schedule.json`: `daily` at `02:00`, `Asia/Kolkata`	file:ap_pipeline/src/ap_pipeline/schedule.json; feat.tasks (TSK-01)
Existing tests	`tests/test_pipeline.py`: `test_run_maps_and_rejects`, `test_paid_flag`, `test_missing_column_rejected`	file:ap_pipeline/tests/test_pipeline.py
Synthetic data	`data/sample_feed.csv` has header `invoice_id,vendor,due_date,amount,paid` (no `paid_date` column)	file:ap_pipeline/data/sample_feed.csv
Files TSK-01 may change	`src/ap_pipeline/transform.py`; `src/ap_pipeline/__init__.py` (version step only); `tests/test_pipeline.py`	feat.tasks (TSK-01)
Files TSK-02 will add	New `src/ap_pipeline/delays.py`; new `tests/test_delays.py`	feat.tasks (TSK-02)
Must not change	`ingest.py`, `dq.py`, `pipeline.py`, `schedule.json`	feat.tasks (TSK-01)
CI static checks	No `print`, `eval`, `exec` or bare `except`; lines at most 140 characters	file:ap_pipeline/ci.py
Commit convention	Commit messages start with the story ID (for example `STORY-01`)	org.engineering_stack; record:memory; feat.tasks (section 2)
Push	Done by the spine after evaluation, not by this agent	record:brief

3. Notes and assumptions

ID	Note
RA-1	Checks were made by reading the checkout's files and the step's tool view. This step does not run shell commands, so this agent did not run `git status` or a push. The clean tree and write access come from tool:repo-access.
RA-2	`.git/config` sets an upstream for `main` and `dev` only. `feature/story` has no upstream yet; the spine is expected to set it when it first pushes (file:ap_pipeline/.git/config).
RA-3	TSK-01 and TSK-02 are both in ap_pipeline. This check assumes they share the one branch `feature/story` in sequence (TSK-02 depends on TSK-01, feat.tasks section 1). feat.tasks names the pattern `feature/<story>`, so the Data engineer may prefer one branch per story.
RA-4	`ci.py` imports the third-party package `coverage` (file:ap_pipeline/ci.py). This step did not check that `coverage` is installed for `/usr/bin/python3`.
RA-5	`data/sample_feed.csv` has no `paid_date` column (feed v2 shape). TSK-01 builds on synthetic v3 rows in its tests (feat.tasks TSK-01).

4. Open questions (non-blocking)

ID	Question	Interim answer
RAQ-1	Should TSK-01 (STORY-01) and TSK-02 (STORY-02) use one branch, `feature/story`, or one branch per story (`feature/STORY-01`, `feature/STORY-02`)?	Use `feature/story` for both, in order: TSK-01 first, then TSK-02
RAQ-2	Is the `coverage` package installed for `/usr/bin/python3` in the CI environment?	Assume yes (agent judgement; not checked here); the first CI run in DE-08/DE-09 will confirm it

Tool results (recorded by the spine from the real tools)

[Tool result] [ap_pipeline] Branch feature/story at 3fcbe1b; write access confirmed